



Tratamento de erros e loading

Desenvolvimento Móvel

Aula 18/36

Api

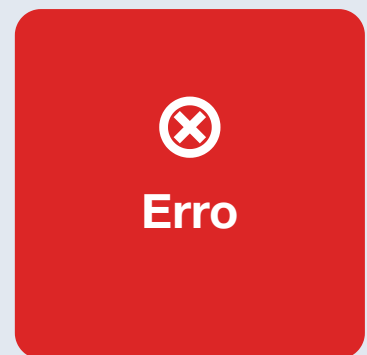
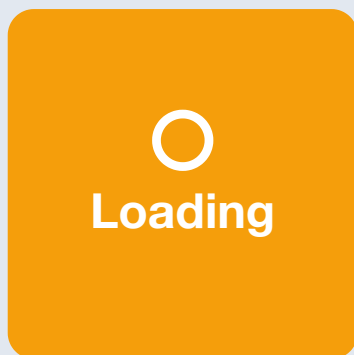
Json

Future

Async

Conceito

Toda tela que consome API precisa gerenciar quatro estados: idle (inicial), loading (carregando), success (dados) e error (falha). FutureBuilder é a solução nativa: recebe um Future e um builder que recebe AsyncSnapshot com `connectionState` (none, waiting, active, done), `data` e `error`. Para maior controle, usa-se um enum `ViewState { idle, loading, success, error }` em conjunto com `ChangeNotifier`. No estado error, exiba mensagem amigável (não erros técnicos) com botão de retry. Shimmer effect (pacote shimmer) oferece animação de placeholder mais elegante que `CircularProgressIndicator`. Tratamento de erros deve incluir: timeout, sem internet (`SocketException`), servidor offline (500), não encontrado (404) e JSON mal formatado (`FormatException`).



FutureBuilder

Shimmer effect



Atividade Prática

Implemente uma tela que busca dados de uma API com delays simulados: exiba CircularProgressIndicator centralizado enquanto carrega, os dados em ListView ao sucesso, e um Container com icone de erro, mensagem "Nao foi possivel carregar" e botao "Tentar novamente" em caso de erro. Use um ChangeNotifier com ViewState. Adicione shimmer effect como alternativa mais elegante ao loading.



Pesquisa

Pesquise o padrao Either da biblioteca dartz para tratamento funcional de erros em Dart: Left para erro, Right para sucesso. Como ele elimina a necessidade de try/catch e torna o fluxo de erros explicito nos tipos? Compare com excecoes tradicionais.



Requisitos

Flutter SDK. Adicionar shimmer ao pubspec.yaml para efeito de loading.